Final Project: Blog Application (NestJS, MongoDB, Auth & Media Upload)

Overview
Build a Blog Application using NestJS, MongoDB and Mongoose with authentication, authorization, media uploads and RBAC.

Core Features
• Authentication: Register, Login, Password Hashing, JWT, Protected Routes.
• Users CRUD.
• Posts CRUD (Create, Update, Delete, Get All, Get User Posts, Global + Allowed Group Posts sorted by createdAt).
• Image Upload: User Profile Image using ImageKit, Post Images using ImageKit, Store URLs.
• Groups: Create Groups, Admins & Members, Permissions.
• Post Ownership: Users manage only their posts, Super Admin overrides.

Middleware Requirements
• Authentication (JWT)
• Authorization (restrictTo)
• Multer + uploadOnImageKit

Validation
Validate POST / PUT / PATCH requests using Joi or another validation library.

Database Design
User: username, email, password, profileImage, role, timestamps.
Post: title, content, images[], author, group, timestamps.
Group: name, admins[], members[], permissions, timestamps.

API Requirements
Auth: POST /auth/register, POST /auth/login.
Users: CRUD + Upload User Profile Image.
Posts: CRUD + Upload Images.
Groups: Create, Add/Remove Users, Manage Permissions.

Business Logic
Only admins manage group users.
Only allowed users post in groups.
Only owner edits/deletes post.
Super admin overrides all rules.

Deployment
1. GitHub Repository
2. Vercel Deployment

Bonus
• Upload User Profile Image using ImageKit.
• Upload one or more images for Posts using ImageKit.

Expected Outcome
Production-like backend, Authentication, Authorization, Media Upload, RBAC, Clean Architecture.

Notes

Follow clean code principles, MVC architecture, handle edge cases, and test using Postman.